

Abstract

As the capabilities of frontier AI systems rapidly advance, there is growing interest in a multilateral agreement between nations to mitigate the risks posed by future, powerful systems. Any such agreement will require a quantifiable definition of what counts as a "frontier" model. Existing legislation, like the EU AI Act and California's SB 53, uses the number of floating-point operations (FLOPs) in the training run as the threshold for which models count as frontier, and proposals for international agreements extend this framing. Today, these thresholds rest entirely on AI developers' self-reports. An international agreement between rival nations cannot assume compliance, so a treaty will be contingent on a technical means of estimating the number of FLOPs in a training run that can be performed by a verification team independent from the AI developers. In this work, I stress-test FLOP estimation on a dual-GPU Nvidia V100 node to develop a minimum viable product (MVP) for how FLOP estimation could work in a real data center. Following prior experiments on an Nvidia Jetson Orin Nano, a verifier ("blue team") with system-level access, but no visibility into workload code, estimates training FLOPs from external signals: GPU power draw, GPU memory bandwidth utilization, and GPU interconnect traffic. Calibrated against sample LLM pre-training workloads spanning 88 hyperparameter configurations, the estimator achieves 10.4% median absolute error on held-out workloads against a ground truth computed by PyTorch's FlopCounterMode library. Then, assuming the role of a non-compliant party, I design and run adversarial workloads that increase the estimator's median error to ~25% under the strongest strategy, with the worst configuration under-reporting by 41%. The result is an MVP for adversarially-tested FLOP verification.

Introduction

Motivation

For want of a quantifiable way to decide what counts as a frontier AI model, compute thresholds have emerged as the standard for AI policy: [California's SB 53](#)[1] uses 10^{26} floating-point operations (FLOPs) in the training run as the threshold for what counts as a frontier model and the [EU AI Act](#)[2] applies the same categorization at 10^{25} . Proposals for international AI agreements[3][4][5] extend the use of training FLOPs to determine part or all of the threshold for what counts as a frontier model under the agreement. Current AI laws have no way of actually verifying AI companies' claims about the number of FLOPs used in training and instead just rely on self-reports, but an international AI agreement can't assume compliance from each involved party. As such, we'd like to verify the number of FLOPs used in LLM training runs through side-channel GPU readings. This allows AI developers' code and data to remain hidden from the verifiers of the AI agreement, but allows verification of training FLOPs even under conditions where the model training might be adversarially changed to circumvent them. My [earlier work](#)[9] built a Minimum Viable Product (MVP) for how this verification could work on an Nvidia Jetson Orin Nano. In this work, I extend the MVP to work on a dual-

V100 node with NVLink interconnect, add NVLink data transfer size as an additional input to the FLOP estimator, and conduct red-teaming using adversarial workloads to expose weaknesses in the estimator.

Related Work

[EpochAI](#)[6] has done work on estimating the number of training FLOPs using (1) insider knowledge about the model architecture and (2) open-source reports of the number of accelerators, amount of time used in training runs, and expected utilization rate. My estimator extends their method (2) to include power-monitoring and real-time data on utilization.

[Chaudhuri et al.](#)[7] demonstrate how thermal and power side-channels, along with assumptions about model architecture, can be used to extract information about transformer model weights during a training run. While an explicit goal of AI treaty verification is to NOT expose model secrets to the verifiers, this shows that extracting information about transformer training runs through side-channels is possible.^{^1}

Recently, [Rahman and Tajdari](#)[8] showed how GPU system-level readings could be used to classify AI workloads as training vs. inference.

Initial FLOP-estimation model

Methodology

Hardware and Observable Signals

All experiments are run on a workstation built around two [Nvidia Tesla V100-SXM2-16GB](#)[10] GPUs, mounted on an SXM2 adapter board and connected to each other by NVLink (six NVLink 2.0 links per GPU at ~25 GB/s per direction each). Using the V100s gives the advantage of using hardware that was, admittedly a few chip generations ago, used in production to train frontier AI systems. The reason for using an outdated chip, like the V100, instead of newer GPUs was so I could have physical access to the hardware for experimenting. In the quest to make the training setup slightly more similar to how frontier AI training data centers work today, I use NVLink interconnect to replicate the typical lowest level of inter-GPU communication used in frontier AI data centers. To estimate FLOPs, I run a sample LLM training script across both GPUs and then observe the following signals:

- **Power (W):** Each V100 reports its total board power through an onboard sensor exposed by nvidia-smi. We poll this reading for each GPU at 2 Hz and sum across the two cards. Unlike the Jetson's shared CPU/GPU/CV rail, this reading covers the GPU board alone, so host CPU power is excluded by construction.

- **GPU Memory Activity (%):** Nvidia's Data Center GPU Manager ([DCGM](#))[11] reports `DRAM_ACTIVE`, the fraction of cycles in which each GPU's memory interface is active. All this tells us is how often the memory controller is active, not *how much* data is being transferred. Still, higher memory activity suggests there is a lot of data moving between the GPU's compute units and its on-card HBM2 memory, which can be indicative of a training run happening.
- **NVLink Interconnect Traffic (bytes):** DCGM also reports per-GPU NVLink ingress/egress byte rates, along with PCIe traffic, which I integrate into cumulative byte counts for each GPU. This observes the size of data that gets transferred during the all-reduce step in a distributed LLM training run where two GPUs which compute a training step in parallel must synchronize their weight updates so they don't diverge from each other.

Constructing Sample Workloads

A pair of V100s is still far too small to run an actual LLM training workload, so as on the Jetson, I use a script that mimics the behavior of a real workload. It initializes a transformer and then runs a training loop for a preset number of steps on randomly generated tokens (step counts are sized per configuration so that every run trains for roughly the same active wall-clock time). Unlike the Jetson version, the script runs a Distributed Data Parallel (DDP) job to distribute that training workload across the two GPUs, where each GPU holds a full replica of the model. After each step, the gradients are averaged between the two GPUs over the NVLink interconnect. It can be configured with a range of hyperparameter settings explained below.

- **Model Width (`d_model`):** the size of the vector representing each token as it flows through the model. A larger model width will quadratically increase the size of matrix multiplications that happen during training.
- **Depth (`n_layers`):** how many identical transformer blocks are there in total? The total number of operations in the training run will scale linearly with the number of blocks.
- **Attention heads (`n_heads`):** How many parallel attention operations are used on the input token vector? This doesn't impact the total number of operations.
- **Feed-forward width (`d_ff`):** the hidden dimension of the two-layer Multi-Layer Perceptron (MLP) in each transformer block
- **Sequence length (`seq_len`):** how many tokens are processed together in parallel? This increases the computation done by the MLP linearly and the attention heads quadratically.
- **Batch size (per GPU):** how many sequences of tokens are processed in a single step on each GPU? Under DDP the global batch is twice the per-GPU value, since both replicas step together. In a real

LLM training setup, changing this would not be expected to have a big impact on FLOPs because although more sequences are processed in parallel, there is typically only a fixed number of tokens for the training run to get through. However, in this experimental setup, the tokens are made up as the training run goes for a fixed number of steps, so we should expect FLOPs to increase linearly with batch size.

- **Optimizer (Adam vs. SGD):** The algorithm used for updating model weights during training. The size of these operations should be dwarfed by the much larger matrix multiplications.
- **Numeric Precision (FP16/FP32):** What precision to represent every number in during the training run? Changing this shouldn't affect the ground truth FLOPs, but might confuse the estimator since an FP16 operation will be less intensive than an FP32 operation. To keep the experiment simple, I stick with the V100's default, FP16. Switching to FP32 would not be advantageous for an adversarial model developer, anyway, as switching to higher precision increases the amount of work the GPU has to do, while keeping algorithmic FLOPs the same.

While the sample training run script can simulate a workload with any value of hyperparameters, we only care about the FLOP estimator being accurate on workloads which emulate a realistic training run that an AI developer might actually do. In practice, we'd expect an AI developer to want to fully utilize their GPUs as best they can, so I define "frontier" sample training runs as ones in which each GPU is active at least 80% of the time on average, as reported by polling `nvidia-smi`. Note, this is different from the memory-activity percentage which is measuring how often the memory interface is running. So the hyperparameter configurations were re-sized upward into the regime where FP16 tensor-core matrix multiplications keep both GPUs saturated. Since I was unsure which hyperparameters to use to be above the 80% threshold, I ran the script under a wide range of configurations, organized into families varying by model width and depth, long sequences, wide feed-forward layers, deep-narrow shapes, Jetson-scale boundary cases, attention-head geometry, and SGD-vs-AdamW optimizer selection, and only kept the ones which, on average, utilized both GPUs above the 80% threshold.

Ground Truth Computation

We can get the ground truth for the number of FLOPs by looking at the shapes of tensors involved in the training algorithm. For example, multiplying an $(m \times k)$ matrix by a $(k \times n)$ matrix requires $m \cdot n \cdot k$ multiplications and roughly as many additions, so we could count this as $2 \cdot m \cdot k \cdot n$ FLOPs towards the ground truth, regardless of how the hardware executes this at the lowest level. To get the ground truth FLOPs for the various sample training runs in my experiments, I used PyTorch's [FlopCounterMode library](#) [12]. This library hooks onto PyTorch to intercept every tensor shape as it is used in low-level operations during the training run and dynamically compute the algorithmic FLOPs as the program executes. Under DDP there is one addition: FlopCounterMode counts the FLOPs of a single rank's local batch, and because both ranks run identical shapes,

the aggregate ground truth is exactly the per-rank count multiplied by the number of steps and the number of GPUs.

The FLOP Estimator

The estimator's algorithm is shown below in equation (1).

$$(1) \text{ TFLOPs} = (E_{\text{net}} - E_{\text{PER_TB}} \cdot \text{TB_moved} - E_{\text{PER_NVL}} \cdot \text{NVL_TB} - P_{\text{overhead}} \cdot t) / E_{\text{PER_TFLOP}}$$

I explain terms below, but first, let me motivate on a high level what this equation is doing. In the simplest case, we would like to just figure out how much energy the GPU is using, calibrate our $E_{\text{Per_TFLOP}}$ parameter based on a bunch of workloads with known total FLOP counts, and then predict on held-out examples based on this GPU energy reading. The issue is that not every joule consumed by the board goes directly towards compute. This is where the additional terms, TB_moved , NVL_TB , and P_{overhead} , come in to represent the energy consumed by memory traffic, NVLink traffic, and by board activity unrelated to the size of the computation, respectively. The result is that after calibrating all parameters, the terms in the numerator should approximate the energy consumption the GPU's compute is directly responsible for.

- **E_{net} (J):** This is the energy derived from power measured by `nvidia-smi`, summed over both GPUs, minus the idle baseline power measured during the calibration phase, integrated over time
- **$E_{\text{PER_TB}}$ (J/TB):** The energy cost of moving one terabyte of data between the GPU's HBM2 memory and its compute units. It is a constant set during the calibration phase.
- **TB_moved :** The total number of terabytes transferred during the workload. This is measured actively during estimation by monitoring the DCGM memory-activity signal and converted to bytes using the HBM2's max bandwidth of 900GB/s.
- **$E_{\text{PER_NVL}}$ (J/TB):** The energy cost of moving one terabyte over the NVLink interconnect between the two GPUs. It is a constant set during the calibration phase.
- **NVL_TB :** The total number of terabytes moved over NVLink during the workload during the all-reduce operations. It is measured actively during estimation by integrating the DCGM TX and RX byte-rate counters over time, summed over both GPUs.
- **t (s):** The total duration of the workload in seconds as determined in the monitoring daemon
- **$E_{\text{PER_TFLOP}}$ (J/TFLOP):** The total energy cost of one TFLOP of GPU compute. In practice, this is whatever energy is left over after accounting for the idle baseline, fixed overhead, and memory

traffic. During estimation, this is a constant that is set during the calibration phase.

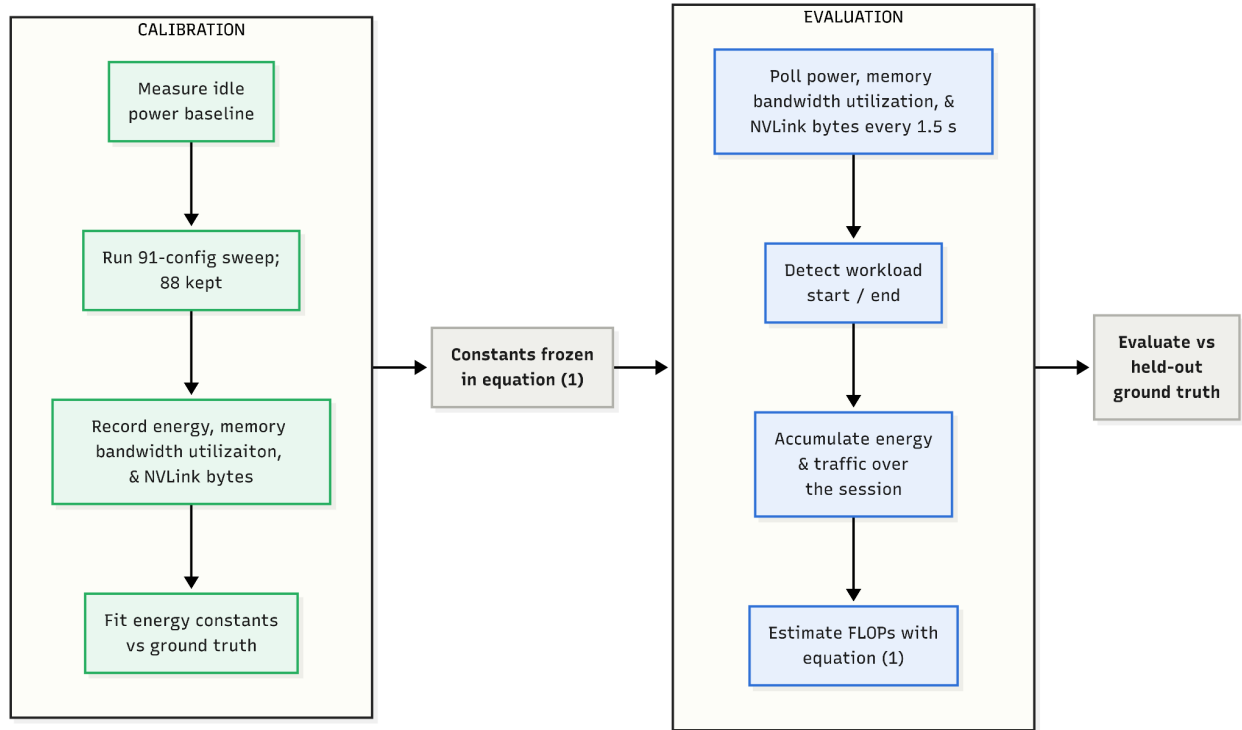
- **P_Overhead (W):** This is the fixed overhead power that we would expect from running any job. The hope is that this will account for the portion of the boards' power draw that is not attributable to compute or memory traffic.

As the results show, the inputs TB_moved and NVL_TB did not end up mattering towards the estimator's accuracy.

Detecting Workloads

In order to estimate the number of FLOPs in a workload, the estimator needs to know when a new workload has started and when it ends. To do this, the estimator is deployed in a daemon which polls for GPU utilization every 1.5 seconds. A new workload starts when GPU utilization is above 5% for two consecutive polls and ends when 3 consecutive polls are below 5%. At the end of a detected workload, the daemon accumulates the power and memory activity logged over the course of the session along with the total time elapsed and outputs a total FLOP estimate for the workload using equation (1). This is then compared against the ground truth given by FlopCounterMode to determine how wrong the estimate was as a percentage of the correct count (error %).

Figure 1



Methodology overview: calibration phase is run first, followed by using the calibrated constants during the deployment phase

Results

I generated 91 sample hyperparameter configurations using the sample LLM training run script. For each trial, I ran the sample workload while collecting traces from the three inputs (GPU power, memory bandwidth utilization, and NVLink traffic size). Since the estimator works by taking the sum of each input upon completion of a workload, there is no need to run it while a workload is actually active when only testing for accuracy: you can just feed it the data collected from running the workload previously.

I based my hyperparameter selections on shape conventions of the classic transformer literature: the original transformer block sets $\text{dim_feedforward} = 4 * \text{d_model}$ [13], the GPT-3 family reports the width/depth/head/batch combinations used in a production-scale training run [14], and the scaling-laws paper finds that model loss depends only weakly on the exact width/depth shape at fixed compute [15]. Ideally, using these formulas for my training run simulator script should result in similar hardware-level behavior as an actual frontier training run, but just to double check, after running the first trial with all 91 workloads, I only kept ones which averaged above 80% GPU utilization. This nullified 3 of the configurations, so I was left with 88 different workloads to continue for 9 more trials, making a total of 10 trials collected for 88 workloads. Table 1 shows the selected hyperparameter values of the remaining workloads.

Table 1

Hyperparameter	Values Used (n=88)
d_model	512, 768, 1024, 1536, 2048
n_layers	3, 6, 12, 16, 24, 32
n_heads	4, 8, 12, 16, 32
head_dim	32, 64, 96, 128, 256
d_ff	2048, 3072, 4096, 6144, 8192, 12288
seq_len	128, 256, 512, 1024, 2048, 4096
batch	2, 4, 8, 16, 32
optimizer	AdamW (n=85), SGD (n=3)

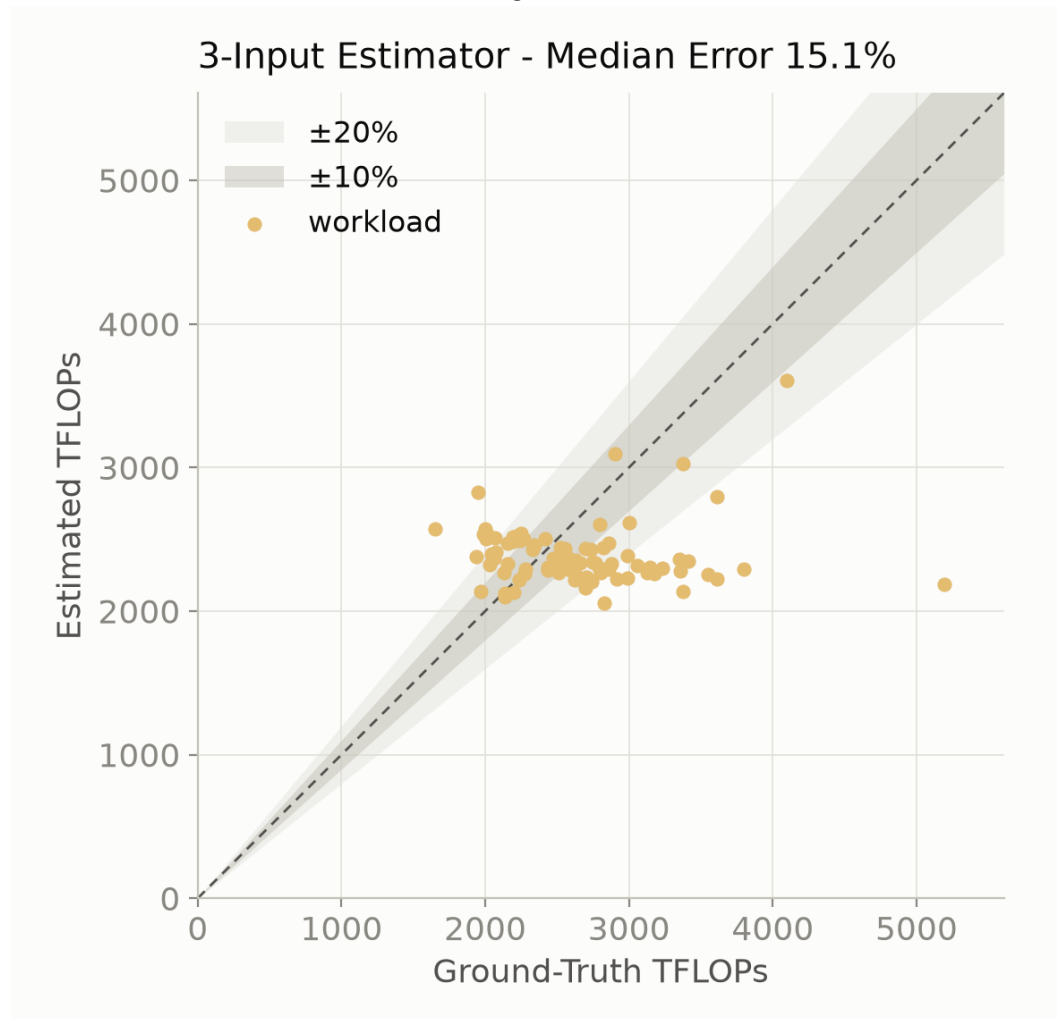
So, the reason I try a bunch of different hyperparameter configurations rather than just picking a couple that most match the papers cited above is to be able to calibrate the estimator on a portion of the workloads and then test it on workloads held-out from calibration to see how it would generalize to arbitrary LLM training runs. I randomly split $\frac{2}{3}$ of the 88 workloads into the calibration set, which is used to fit the parameters in the

estimator, and the remaining $\frac{1}{3}$ into an evaluation set which is used for testing the estimator's accuracy. I run this process 200 times:

- (1) Randomly select the split
- (2) Use all 10 traces of each of the calibration workloads to fit the linear parameters in the estimator formula
- (3) Test the estimator on all 10 traces of the remaining estimation workloads.

Figure 2 below shows the median estimated FLOP counts for each of the 88 workloads across all 200 trials, compared to the ground truth FLOP counts from FlopCounterMode.

Figure 2



Held-out accuracy of the 3-input estimator. Each dot is one of the 88 workloads. Shaded wedges mark $\pm 10\%$ and $\pm 20\%$ of ground-truth. Median absolute error: 15.1%.

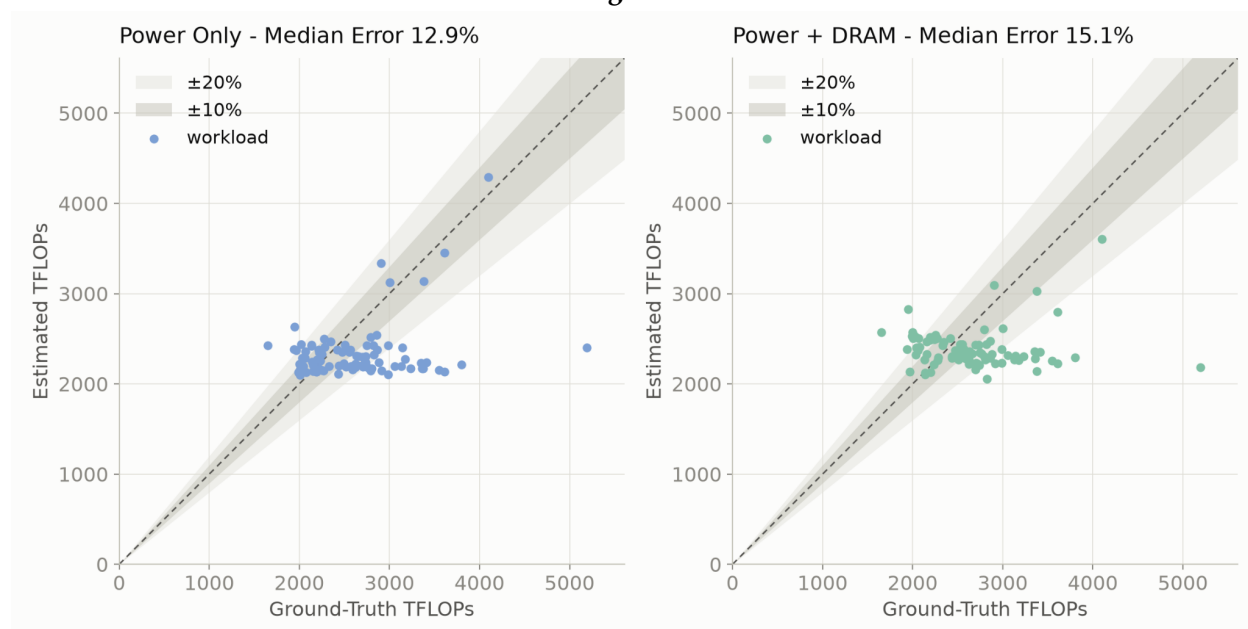
So, over all the 88 workloads, the median amount the estimator was off by, in absolute terms, was 15.1% of the ground truth value. But, as can be seen in the graph, there are many workloads where the estimator was off by more than 20%. I ideally wanted the estimator's median error to be under 10%, as this was a number I was able

to reach in my experiments on the Jetson Orin Nano. I ended up improving upon this median error in two ablation studies.

Improvement #1: Power Is All You Need

First, I found out that the memory bandwidth utilization and NVLink traffic weren't contributing much to the model's output. I had originally thought that including more inputs would improve the accuracy of the estimator since I thought that variation in these two inputs would capture FLOP fluctuations that power alone misses. This turned out not to be the case, as not only were memory bandwidth utilization and NVLink traffic not contributing to accuracy, but it was actually making the estimator worse by including them. Figure 3 shows the estimated vs ground-truth FLOP counts for each of the 88 workloads using power only. I also tested just taking away NVLink traffic and keeping memory bandwidth utilization, but this did not improve accuracy at all.

Figure 3



Ablating inputs under the same protocol as Figure 2. Right: uses both power and memory bandwidth util, but leaves NVLink traffic out (median error 15.1% is the same as with the full estimator). Left: uses power as the only input (median error is 12.9%).

The improvement for each specific workload was marginal, but it did bring down the median error compared to the 3-input estimator from 15.1% to 12.9%.

Now, this is a surprising result, but I think it is surprising in a good way. If all it takes to make accurate FLOP estimates of LLM training runs is monitoring power usage, then this makes the challenge of FLOP estimation a simpler problem: extending this experiment to physical sensors would only require power meters, not

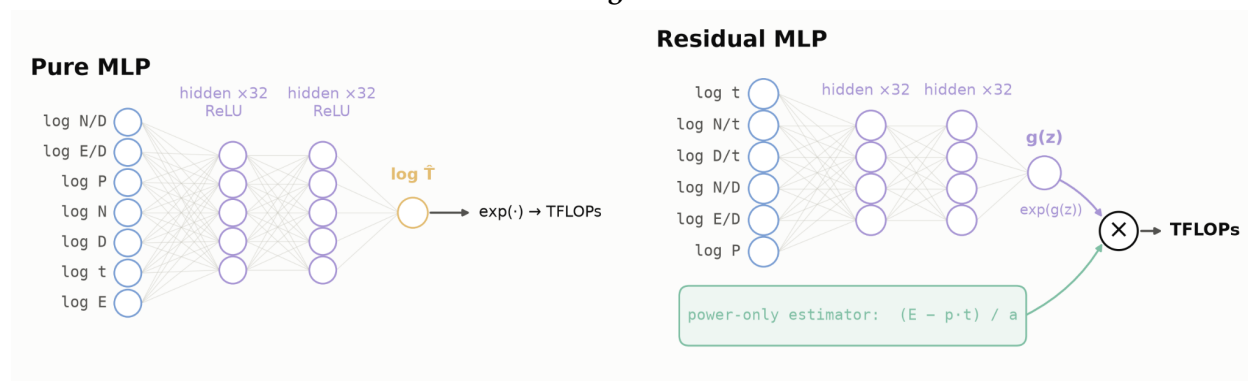
complicated network taps installed on NVLink interconnect or specialized chips that report memory bandwidth utilization.

Improvement #2: Going from a Linear Estimator to an MLP

By reducing the estimator to only use GPU power as the input, I was able to get a 12.9% median error, but this was still above the 10% goal. Initially, my guess was that using a linear estimator would be most suitable to the challenge of FLOP estimation based on the fact that the same approach was able to reach <10% on the Jetson Orin Nano. But, it is possible that doing larger sample workloads distributed across two GPUs means that there are more complex patterns happening in the signal traces during training that a linear estimator is not able to fully capture. Additionally, the field of deep learning has generally found in favor of deeper computation as a tool for improving predictive accuracy.

So, I trained a simple Multi-Layer Perceptron (MLP) on features derived from the same three signals, plus workload duration and a couple ratios between signals, put through a log scale. The features (net energy, duration, TB memory bandwidth, TB NVLink, average power, and various ratios of these) are run through two 32-unit hidden layers. Then, the output is used in two different ways: (1) pure MLP treats the output directly as the FLOP estimate without any additional computation and (2) residual MLP which uses the output as a correction to the power-only linear estimator. Figure 4 shows the architecture of these two approaches.

Figure 4

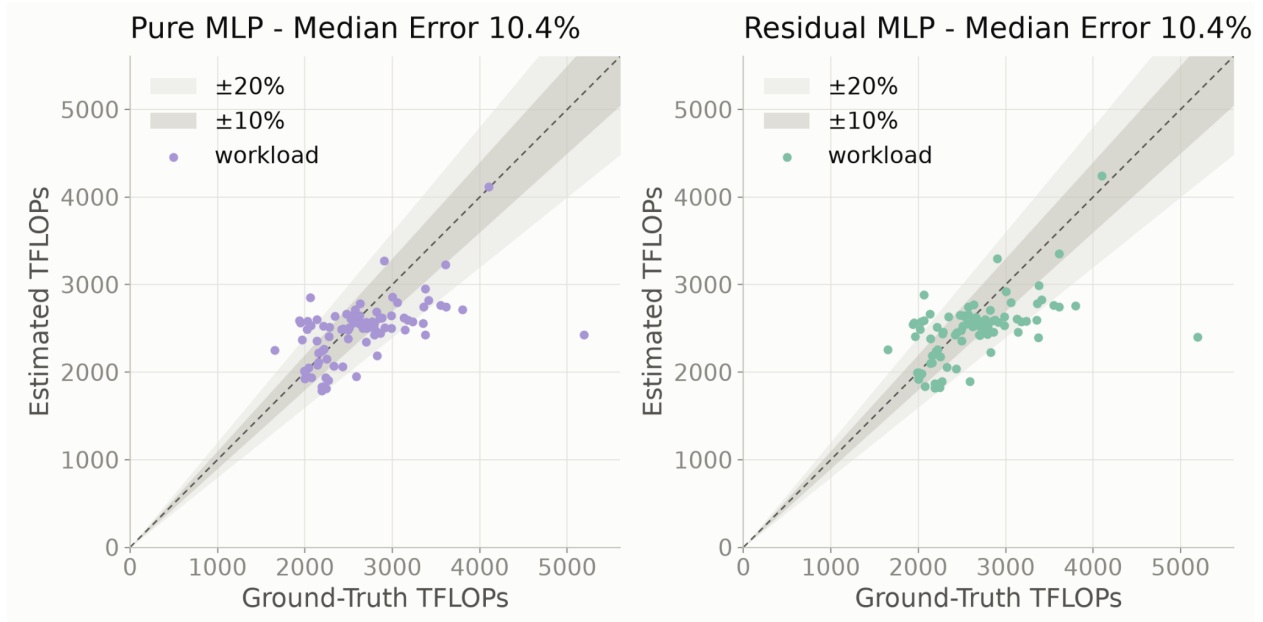


Two versions of the MLP estimator. Left: Pure MLP computes TFLOPs direction. Right: Residual MLP uses output as an adjustment to the power-only estimator. Features are labelled as follows

- P = average power during the workload
- N = TB moved over NVLink
- D = TB moved over memory bandwidth
- t = duration (in seconds) of the workload
- E = total energy consumed by the GPUs during the workload

As it turns out, both methods end up with very similar results, only differing slightly in their estimates, and both get a median error of 10.4%. Figure 5 shows the estimated vs. ground-truth FLOP counts for both methods.

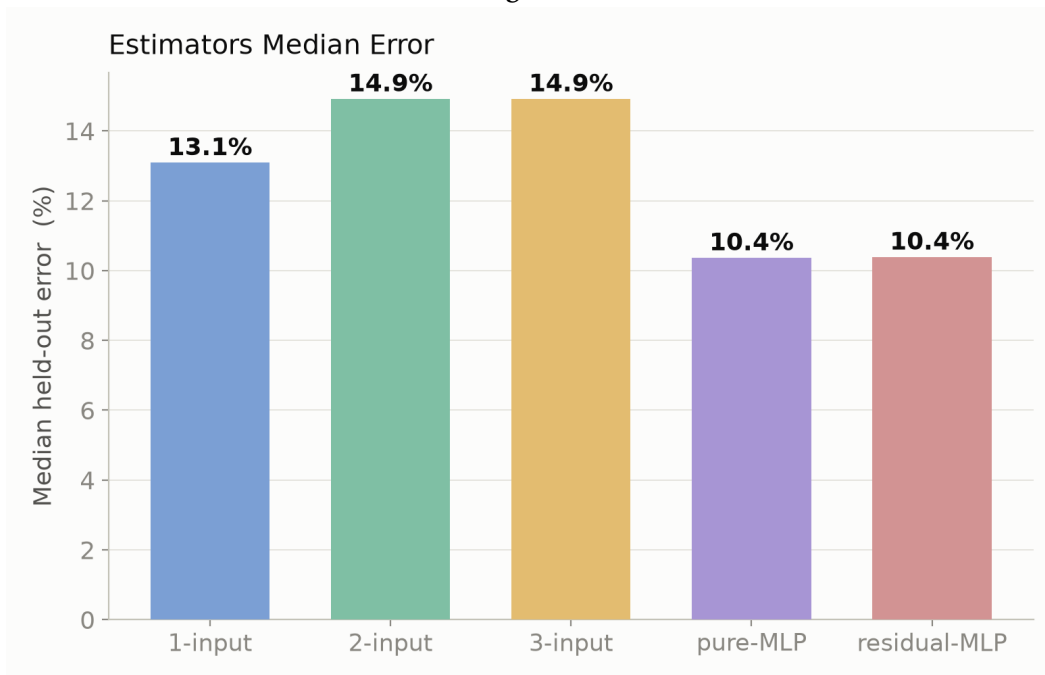
Figure 5



Held-out estimate vs ground truth for the two MLP versions. Each dot is the median held-out estimate for one of the 88 workloads. Both variants reach 10.4% median absolute error.

Figure 6 below gives the overview of the five different estimators that were tested. Both MLP approaches to the estimator achieve the lowest error.

Figure 6



Median held-out error of all five estimator configurations.

Red-Team Adversarial Workloads

Methodology

Up to this point, we've only tested the estimator on benign LLM training workloads. This is a good starting point, but if this method were to be used in a real frontier AI data center to enforce a limit on the size of LLM training runs, it needs to be robust against potentially adversarial training workloads, ones which attempt to bypass the estimator or trick it into under-estimating, employed by the model developers. In this section, I conducted red-teaming experiments to try to increase the error of the estimator and pass adversarial workloads by the monitoring daemon.

First, though, my inspiration for conducting red-teaming on the estimator comes from Capture The Flag games [16] used by the cybersecurity community to find and patch vulnerabilities in systems. In these games, there are separate blue (defending) and red (attacking) teams which both pursue the opposite goals of defending or exploiting a system. Since the red-team tends to employ the same strategies that an actual adversary would use, this gives the defenders additional information on how to harden the system beyond the examples they could think of on their own.

I believe this would be a good format to also test verification systems since having a red-team act as an adversarial model developer gives the builders of the verification mechanism more info on how to make it robust that they couldn't get on their own. Ideally, I would have done this in my experiment, however, time constraints meant I had to play both the roles of the red and blue team.

For this specific project, I define the red-team scope below: the red-team will...

- Have a general knowledge of how the estimator works, but not the exact code
- Will not be able to run workloads that are arbitrarily inefficient. For the sake of setting a limit, I will not allow any red-team strategies that use more than double the GPU-hours of a benign workload when holding ground-truth FLOPs constant.
- Have root-level access to the GPUs, but cannot directly interfere with the signals the estimator is using

Assuming this role, I tested the following five strategies as different ways the red-team might be able to fool the estimator.

1. **Workload Splitting:** Insert 6 second pauses at points during the training run so that the estimator will think each one marks the end of a separate training job.

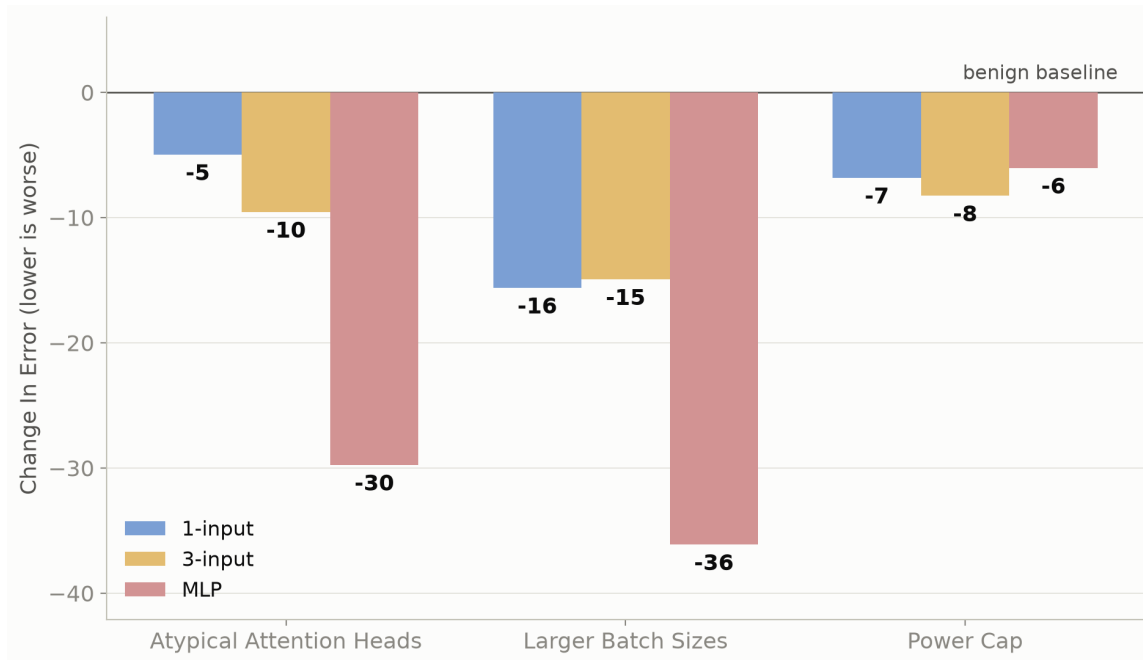
2. **Throttling:** Insert micro-pauses between each training step, making the workload seem less intense in the hope it might not reach the utilization threshold required by the daemon for detection.
3. **Atypical Attention Heads:** The estimator is calibrated on a wide set of hyperparameters, but the ones tested are all based on what would be reasonable values to choose. I alter the number of attention heads (`n_heads`) beyond the range used in the benign workloads.
4. **Larger Batch Sizes:** The estimator was calibrated on batch sizes that make sense for the 16GB of memory in the V100s, but inflating this hyperparameter specifically might allow more FLOPs to get run without as much of an effect on the energy consumption of the GPUs.
5. **Power Capping:** Use system-level commands to run the GPUs at a lower voltage in the hope that an estimator based mostly on the amount of power being used will predict fewer FLOPs if the power consumed cannot exceed a certain threshold.

To test these strategies, I freeze the 3 main configurations of the estimator (3-inputs, power-only-input, and MLP) after calibration on the benign workloads and run the monitoring daemon in the same way as when testing on benign workloads.

Results:

I score every adversarial run against the multiple versions of the estimator, calibrated on the benign workloads only, and compare the error on the estimator. For each adversarial strategy, I use multiple configurations (explained below) to cast a wide net on what might be the best red-teaming approach. Figure 7 below shows the change in median error of strategies 3-5 compared to the benign baseline. Since strategies 1 and 2 are attempting to bypass the monitoring daemon entirely, not affect the estimation, these are withheld from this figure and shown below. The overall result was that, while the MLP version of the estimator was the most accurate on benign workloads, it was also the most gameable in an adversarial setting, showing an increase in error of 36%.

Figure 7

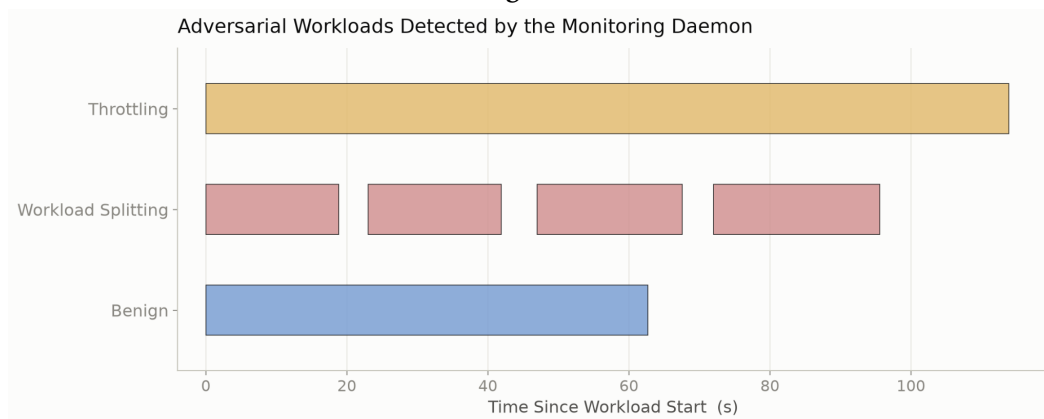


Change in signed error going from benign workloads to each strategy's strongest attack setting.

Workload Splitting and Throttling

Strategies 1 and 2 attempt to bypass the estimator by not having their full workload recognized by the monitoring daemon. For workload splitting, an intermittent pause of 6 seconds is inserted at three points along the training run. For throttling, a micro-pause of a fraction of a second in length is inserted after every training step. Figure 8 shows how the monitoring daemon interpreted these workloads, with workload splitting causing the daemon to incorrectly detect the single workload as four separate ones. For throttling, however, the daemon was not fooled.

Figure 8



How the monitoring daemon attributed the Workload Splitting and Throttling runs of one identical workload, next to a benign control.

One reason why workload splitting worked, but throttling didn't, is that the monitor detects a new workload whenever peak GPU utilization jumps above 5%. With a 6 second pause, this is enough to drop utilization down to 0, but throttling only ends up lowering the average utilization, not the peak, which doesn't change the monitor's behavior.

Out-of-Distribution Training Conditions

Strategies 3, 4, and 5 all try to mess up the estimate by either using atypical hyperparameter values (3 and 4) or capping GPU power (5). For each strategy, I start from an existing benign workload in the evaluation set, and just change one parameter at a time as shown in Table 2.

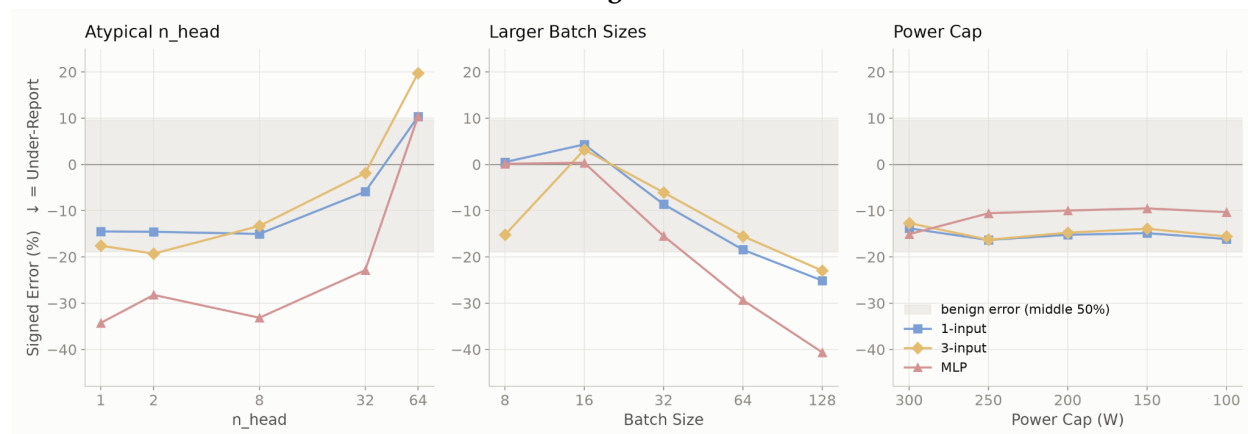
Table 2

Strategy	Values Held-Constant	Value Changed
Atypical Num. Attention Heads	• d_model = 1024 • seq_len = 512 • batch = 16 • n_layers = 12 • dim_feedforward = 4096 • FP16 • AdamW	n_head = {1, 2, 8, 32, 64}
Larger Batch Size	• d_model = 1024 • seq_len = 256 • n_layers = 6 • n_head = 8 • dim_feedforward = 4096 • FP16 • AdamW	batch_size = {8, 16, 32, 64, 128}
Power Capping	• d_model = 1024 • seq_len = 512 • batch_size = 16 • n_layers = 12 • n_head = 8 • dim_feedforward = 4096 • FP16 • AdamW	power cap = {300, 250, 200, 150, 100} W

Figure 9 below shows the change in estimator error across different values for all three strategies. Power capping ended up not being an effective adversarial strategy as the error stayed within the benign error bounds and

didn't change at all under different caps. Changing `n_head` and batch size, however, did result in outlier errors at either end of the extreme for the tested values. Given that these adversarial workloads were still within a similar efficiency as their benign counterparts, it is a concern that these values could be selected by the adversarial model developer but additional testing would be needed to confirm if these values would result in model convergence under a reasonable time constraint if an actual training run was conducted.

Figure 9



Signed estimation error for every S3 (atypical attention heads), S4 (batch inflation), and S5 (power cap) configuration, under each of the three frozen estimators. The grey band is the range of signed errors observed on benign held-out workloads.

Discussion/Conclusion

FLOP estimation is a prerequisite for verifying an international agreement that wishes to limit pushing the frontier of AI training by restricting the use of compute. Since current AI legislation using FLOPs to define frontier training runs relies on self-reports, there is a lack of technical mechanisms which could provide reliable estimates of how many FLOPs a training run consumes without having to analyze the code which AI developers sensibly wish to keep secret. In this paper, I present an estimator and monitoring daemon which can run in the background while a GPU workload is active to estimate the algorithmic FLOPs of benign, sample LLM training runs with a median error of 10.4%. Additionally, I perform red-teaming experiments on both the estimator and monitoring daemon and show that the daemon can be easily fooled by intermittent pausing inserted into the workload by an adversarial model developer and that most adversarial strategies fail to significantly increase the estimator's error apart from batch size inflation. While this red-teaming process exposes critical failures of the estimator and daemon that would need to be improved before use in an actual frontier AI verification regime, I note that a limitation of this process was that I played both sides of the red/blue-team experiment, and thus knew more details about how the estimator and monitoring logic works.

Future Work

My choice of hardware was due to budget constraints and a desire to have an on-site node to potentially make use of side-channel sensors as additional inputs to the estimator. I ended up shifting away from using physical sensors and relied only on system-level readings, so if I were to do this experiment again, I would use newer and more GPUs that I rent as bare-metal nodes from a cloud provider. Additionally, there now exist dedicated clusters [17][18] for use in verification mechanisms research which were not available at the time of starting this project. Improving FLOP estimation will require performing experiments in more realistic settings to how frontier AI models are actually trained: many up-to-date GPUs with high-speed interconnect.

The future work I am planning on working on most immediately, however, is using the same red/blue team, iterative testing approach I used to stress-test the FLOP estimator in this experiment for verifying whether an AI workload is training or inference. While FLOP estimators based on hardware-level signals will likely not ever perform at 100% accuracy, inference and training runs have distinguishable patterns in the inter-GPU communication that happens during the workload.

Reflecting on the direction of the nascent field of prototyping mechanisms for international AI agreement verification, I anticipate that the best approach will require a Swiss-Cheese model [19] of detection. I believe a verification regime which relies solely on FLOP estimation to determine whether a training run is allowed or not places too much weight on a single point of failure by relying fully on an imperfect detection mechanism. I think future work in this area should explore a variety of possible approaches.

References

- [1] California Senate Bill 53, Transparency in Frontier Artificial Intelligence Act (2025). https://leginfo.ca.gov/faces/billTextClient.xhtml?bill_id=202520260SB53
- [2] Regulation (EU) 2024/1689 (EU AI Act), Article 51. <https://artificialintelligenceact.eu/article/51/>
- [3] Y. Shavit. What does it take to catch a Chinchilla? Verifying Rules on Large-Scale Neural Network Training via Compute Monitoring. arXiv:2303.11341, 2023. <https://arxiv.org/abs/2303.11341>
- [4] A. R. Wasil, T. Reed, J. W. Miller, and P. Barnett. Verification methods for international AI agreements. arXiv:2408.16074, 2024. <https://arxiv.org/abs/2408.16074>
- [5] A. Scher and L. Thiergart. Mechanisms to Verify International Agreements About AI Development. arXiv:2506.15867, 2025. <https://arxiv.org/abs/2506.15867>
- [6] Epoch AI. Estimating training compute of deep learning models. 2022. <https://epoch.ai/blog/estimating-training-compute>

- [7] A. Chaudhuri, S. Shukla, S. Bhattacharya, and D. Mukhopadhyay. "Energon": Unveiling Transformers from GPU Power and Thermal Side-Channels. ICCAD 2025; arXiv:2508.01768. <https://arxiv.org/abs/2508.01768>
- [8] R. Rahman and S. Tajdari. Detecting Hidden ML Training With Zero-Overhead Telemetry. arXiv:2606.19262, 2026. <https://arxiv.org/abs/2606.19262>
- [9] W. Fowler. Estimating LLM Training FLOPs on the Nvidia Jetson Orin Nano. <https://www.lesswrong.com/posts/wZpXEWgiG7k98p6RK/estimating-llm-training-flops-on-the-nvidia-jetson-orin-nano> July 2026.
- [10] NVIDIA. NVIDIA V100 Tensor Core GPU Datasheet (Tesla V100-SXM2). <https://images.nvidia.com/content/technologies/volta/pdf/volta-v100-datasheet-update-us-1165301-r5.pdf>
- [11] NVIDIA. Data Center GPU Manager (DCGM) Documentation. <https://docs.nvidia.com/datacenter/dcgml/latest/>
- [12] PyTorch, torch.utils.flop_counter.FlopCounterMode. https://github.com/pytorch/pytorch/blob/main/torch/utils/flop_counter.py
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention Is All You Need. NeurIPS 2017; arXiv:1706.03762. <https://arxiv.org/abs/1706.03762>
- [14] T. Brown et al. Language Models are Few-Shot Learners. NeurIPS 2020; arXiv:2005.14165. <https://arxiv.org/abs/2005.14165>
- [15] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling Laws for Neural Language Models. arXiv:2001.08361, 2020. <https://arxiv.org/abs/2001.08361>
- [16] Wikipedia. Capture the flag (cybersecurity). [https://en.wikipedia.org/wiki/Capture_the_flag_\(cybersecurity\)](https://en.wikipedia.org/wiki/Capture_the_flag_(cybersecurity)) (accessed August 2026)
- [17] Amodo Design. The Side Channel Cloud (instrumented 32-GPU verification-research cluster). <https://amododesign.com/ai-verification/> · <https://www.sidechannel.cloud>
- [18] Lucid Computing. Lucid Labs experimentation cluster (instrumented bare-metal GPU infrastructure for AI verification research). <https://lucidcomputing.ai/experimentation-cluster>
- [19] Wikipedia. Swiss cheese model. https://en.wikipedia.org/wiki/Swiss_cheese_model (accessed August 2026)

Thanks to Prof. Shabin Tajik, Madeleine Hoffman, and Daniel Ben-Levi for their feedback on an earlier version of this report.

This work is supported by the University of Chicago Existential Risk Laboratory